

Planificación de un bucle aplicando técnicas de desenrollado en el MIPS segmentado

Queremos ejecutar el siguiente bucle en nuestro MIPS segmentado con **gestión de los saltos en la etapa D**, y sin unidad de detención ni anticipación para la gestión de riesgos. Es decir, el procesador no para nunca, y todos los datos se leen del banco de registros. Es una ISA, con instrucciones retardadas. El compilador/programador debe añadir instrucciones “nop” o planificar estáticamente el código (reordenarlo) para que las consumidoras consuman el dato que esperan de la productora de la que dependen y para que el salto se ejecute correctamente.

- a) Dibujar el grafo de dependencias (dependencias directas, antidependencias y dependencias de salida)
- b) **VERSIÓN 1.** Incluye las nops necesarias y haz un cronograma de la ejecución de una iteración del bucle más la primera instrucción de la 2ª iteración. Usa la siguiente notación para las etapas: **F D E M W** (ver ejemplo en la siguiente página). ¿Cada cuantos ciclos comienza una nueva iteración? Si el bucle se ejecuta 100 veces, ¿cuántos ciclos tardará?
- c) **Completa el segundo bucle** (ver comentarios en rojo), en el que el compilador ha aplicado una de las técnicas clásicas de optimización de código: **desenrollar** (en este caso, grado 2). Esta técnica reduce el número de iteraciones, haciendo el doble de operaciones en cada iteración. De esta forma se reduce la sobrecarga del bucle, y se consiguen más instrucciones para planificar. Por supuesto, se puede aplicar con grados mayores, aumentando cada vez el tamaño del bucle, y reduciendo las iteraciones necesarias.
- d) **VERSIÓN 2.** Incluye las nops necesarias en el segundo bucle y haz un cronograma de la ejecución de una iteración del bucle más la primera instrucción de la 2ª iteración. ¿Cada cuantos ciclos comienza una nueva iteración? Si el bucle original (el INI) se ejecutaba 100 veces, ¿cuántos ciclos tardará la versión desenrollada del INI_2?
- e) **VERSIÓN 3.** Reordenar el código del segundo bucle para minimizar el número de nops necesario y simular ciclo a ciclo la primera iteración del bucle. ¿Cada cuantos ciclos comienza una nueva iteración? Si el bucle original (el INI) se ejecutaba 100 veces, ¿cuántos ciclos tardará la versión desenrollada y reordenada del INI_2?
- f) **VERSIÓN 4.** Volver al código inicial (INI sin nops). Haced un cronograma esta vez para un MIPS con UD y UA visto el viernes (kill de la siguiente instrucción en beq tomado, anticipación M->Ex y W-> Ex, paradas en lw-uso y BEQs sin anticipación de operandos). Si haces uso de la red de anticipación, indica el origen y el destino. ¿Cada cuantos ciclos comienza una nueva iteración? Si el bucle se ejecuta 100 veces, ¿cuántos ciclos tardará?
- g) **Calcula el Sup** de las versiones 2, 3 y 4 con respecto a la 1.

```

INI: LW      R8, 0(R1)
      ADD     R8, R8, R4
      SW      0(R1), R8
      ADDI     R1, R1, #4 // R1 <= R1+4
      SUBI R2, R2, #1 // R2 <= R2-1
      BEQ     R2, R0, INI

```

```

INI_2:  LW  R8, 0(R1)
        ADD R8, R8, R4
        SW  0(R1), R8
        LW  R9, 4(R1) //¿Por qué no se usa R8?
        ADD R9, R9, R4
        SW  4(R1), R9
        ADDI     R1, R1, #¿? // Completar
        SUBI R2, R2, #¿? // Completar
        BEQ R2, R0, INI_2

```

Ejemplo de cronograma:

	1	2	3	4	5	6	7
INI: LW R8, 0(R1)		F	D	E	M	W	
NOP			F	D	E	M	W
NOP				F	D	E	M
...							